

Programming Blue3: An SMT solver for Caprice-Lang

Blue3 is an SMT solver for JHU’s `caprice-lang`. It is used by its typechecker known as the Concolic Evaluator, or `ceval`.

Before Blue3, `ceval` used Z3 to solve SMT formulas. `ceval` still uses Z3, but is now used as a fallback rather than being the sole solver for when Blue3 cannot solve the formula.

Z3 is more than capable of solving our formulas, of course, but the JHU Programming Language lab felt it was overkill for many of the cases. For instance, `ceval` might output something like:

$$(6 \leq a) \wedge (a < 0)$$

We say this formula is **unsatisfiable**, or UNSAT, because a can’t be 6 or more while also being less than 0.

The non-insignificant overhead of calling Z3 made an in-house solver for trivial cases seem like a promising way to improve `ceval`’s performance.

Intro

This report introduces Blue3, a minimal SMT solver for `caprice-lang`. Although small, Blue3 implements a full solver pipeline using modern SAT/SMT techniques. In benchmarks, Blue3’s frontend was just over 60% faster than Z3 on simple formulas.

avg_blue3	avg_z3
222.0 μ s	329.0 μ s

When Blue3 cannot solve a formula, it falls back to Z3. This adds about 20.24 μ s of overhead on average, or roughly 5% compared to calling Z3 directly.

num_slow_cases	avg_slower_by	avg_percent_slower
38	20.24 μ s	4.59%

This is a reasonable tradeoff: Blue3 solves simple formulas much faster, while still using Z3 as a backup.

Before discussing Blue3 itself, we first need some context on $P = NP$, SAT, and 3SAT.

P = NP and Boolean Satisfiability

Oversimplifying, $P = NP$ asks:

If we can check a solution quickly, can we also find that solution quickly?

Some problems are easy to solve and easy to check. For example, sorting a list like:

[5, 2, 9, 1]

into:

[1, 2, 5, 9]

can be done efficiently. Problems solvable in polynomial time are in the class P .

Other problems are harder to solve but easy to verify. Sudoku is the classic example: finding the solution may require search, but checking a completed board is straightforward. Problems whose solutions can be verified in polynomial time are in NP .

So another way to state $P = NP$ is:

If a solution can be verified in polynomial time, can it also be found in polynomial time?

We do not know the answer. Most computer scientists believe $P \neq NP$, but no one has proven it.

This matters because some problems are **NP-complete**: they are in NP , and every other problem in NP can be reduced to them in polynomial time. If we found a polynomial-time algorithm for one NP-complete problem, then every problem in NP could be solved efficiently.

That would have massive consequences for optimization, science, and cryptography.

3SAT and Boolean Satisfiability

3SAT asks:

Given a propositional formula in CNF with at most 3 literals per clause, is there some assignment that makes it true?

A formula is **satisfiable** if at least one assignment makes it true. For example:

$$(p \vee q) \wedge (\neg p \vee \neg q)$$

is satisfiable because this model works:

```
{  
  "p": true,  
  "q": false  
}
```

Plugging those values in gives:

$$(\text{true} \vee \text{false}) \wedge (\neg \text{true} \vee \neg \text{false})$$

which evaluates to true.

But this formula is unsatisfiable:

$$(p \vee q) \wedge (\neg p \vee q) \wedge (p \vee \neg q) \wedge (\neg p \vee \neg q)$$

No assignment of p and q can make every clause true at once.

3SAT is important because it is NP-complete. So if we could solve 3SAT in polynomial time, we would prove $P = NP$.

Blue3 obviously does not solve $P = NP$. Instead, it uses practical SAT/SMT techniques to solve many real formulas efficiently.

SMT solvers extend SAT by allowing richer theory constraints. For example:

$$(6 \leq a) \wedge (a < 0)$$

is not purely propositional, because it talks about integer inequalities. Blue3 maps theory atoms like these into propositional variables:

$$p \wedge q$$

where:

- p represents $(6 \leq a)$
- q represents $(a < 0)$

The SAT solver handles the propositional structure, while the theory solver checks whether the underlying constraints are actually consistent.

Useful Terminology

A **formula** is a boolean-valued expression. In this report, unless stated otherwise, “formula” usually means a formula in **CNF**.

A formula in **CNF** is an AND of clauses:

$$(p \vee q \vee r) \wedge (s \vee t \vee \neg u)$$

A **clause** is one OR-group:

$$(p \vee q \vee r)$$

A **literal** is an atom with a sign. For example:

$$p$$

and:

$$\neg p$$

are both literals.

An **atom** is the unsigned condition underneath a literal. So p and $\neg p$ refer to the same atom: p .

In SMT, atoms can be theory constraints, such as:

$$a = 1$$

or:

$$a \neq 1$$

We will use three main formula categories:

1. A **SAT formula** is purely propositional:

$$p \wedge q$$

2. A **theory formula** is handled by a theory solver:

$$(6 \leq a) \wedge (a < 0)$$

3. An **SMT formula** combines propositional logic with theory constraints:

$$(6 \leq a) \wedge (a < 0) \wedge (r \vee s)$$

Finally, a **solver** takes a formula and returns either **SAT**, usually with a satisfying model, or **UNSAT**, meaning no satisfying assignment exists.

Difference Logic

Recall that Blue3 is meant to handle formulas that are too simple to justify calling Z3, such as:

$$(6 \leq a) \wedge (a < 0)$$

The first challenge was deciding what “simple” meant for `caprice-lang` formulas. Since its formula AST only works over `bools` and `ints`, we focused on the integer-heavy formulas that appeared often in our benchmarks:

formula_id	formula
9	$(0 < a) \wedge ((a + 1) \leq a)$
8	$(0 < a) \wedge ((a + 1) \leq 1)$
56	$(\text{not } (a = 0)) \wedge ((a + 10) = 0)$
11	$(1 < a) \wedge (a < 0)$
88	$(0 < a) \wedge (a < 1)$

This led us to **Integer Difference Logic**, or **IDL**, a fragment of linear integer arithmetic where constraints compare the difference between two integer terms. In SMT-LIB terms, IDL is a sub-logic of Linear Integer Arithmetic over the `Ints` theory.

More formally, an IDL solver decides satisfiability for literals shaped like:

$$(x \leq y) <> c$$

Here, x and y are integer variables or the constant 0, c is an integer constant, and $<>$ is one of $<$, $\leq$, $>$, $\geq$, or $=$. IDL does **not** directly handle $\neq$, nor sums like $x + y$ or any other operator other than $-$ for that matter.

Many of our “simple” formulas fit this difference form, including:

$$(6 \leq a) \wedge (a < 0)$$

We can rewrite it as:

$$((0 - a) \leq -6) \wedge ((a - 0) \leq -1)$$

This rewriting looks unnecessary to us because we can immediately see the contradiction: a cannot be both at least 6 and less than 0. But IDL gives our computer a precise way to recognize that contradiction mechanically.

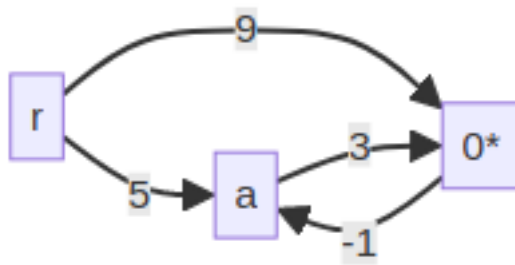
It turns out there is a natural graph interpretation of IDL, where each difference constraint becomes an edge, and satisfiability can be checked with a shortest-path algorithm.

Bellman-Ford

Bellman-Ford finds the shortest paths from one source node to every other node in a directed weighted graph. It also detects **negative cycles**, which are cycles whose total weight is negative.

For example:

```
let simple_no_neg =  
  [ ("a", "0*", 3) ; ("0*", "a", -1)  
    ; ("r", "0*", 9) ; ("r", "a", 5)  
  ] in  
print_mermaid_lr ~id:"simple-no-neg" simple_no_neg;
```



Running Bellman-Ford from r gives:

Node	Distance
a	5
0^*	8

The shortest path to a is:

$$r \rightarrow a$$

with cost 5. The shortest path to 0^* is:

$$r \rightarrow a \rightarrow 0^*$$

with cost:

$$5 + 3 = 8$$

This beats the direct edge $r \rightarrow 0^*$, which costs 9.

The cycle between a and 0^* has cost:

$$3 + (-1) = 2$$

Since the cycle is positive, looping only makes paths more expensive. So there is **no negative cycle**.

Now change the edge $0^* \rightarrow a$ from -1 to -4 :

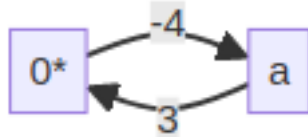
```
let simple_neg =
  [ ("a", "0*", 3) ; ("0*", "a", -4)
    ; ("r", "0*", 9) ; ("r", "a", 5)
  ] in
print_mermaid_lr ~id:"simple-neg-mermaid" simple_neg;
print_bellman_ford ~label:"simple-neg-bf" ~src:"r" simple_neg
```

Now the cycle cost is:

$$3 + (-4) = -1$$

Each loop makes the path cheaper, so there is no true shortest path. Bellman-Ford reports:

Negative cycle found!



Relaxing Edges Bellman-Ford repeatedly loops over every edge and tries to improve the known distance to each node. This update step is called **relaxation**.

An edge is relaxed when:

$$dist[from] + cost < dist[to]$$

In code:

```
let relax_edge tbl was_updated edge =
  let from_, to_, cost = edge in
  match Hashtbl.find tbl from_, Hashtbl.find tbl to_ with
  | (Some du, _), (None, _) ->
    set_distance to_ tbl ~min:(du + cost) ~pred:edge
  | (Some du, _), (Some dv, _) when du + cost < dv ->
```

```

    set_distance to_ tbl ~min:(du + cost) ~pred:edge
  | _ -> was_updated

```

The table starts with every node at infinity, represented by `None`, except the source, which starts at distance 0.

```

let create_tbl ~src edges =
  let tbl =
    edges
    |> to_node_list
    |> ...
  in
  Hashtbl.replace tbl src (Some 0, None);
  tbl

```

Bellman-Ford relaxes all edges at most $N - 1$ times, where N is the number of nodes:

```

let relax_edges edges dist i =
  if i >= Hashtbl.length dist - 1 then `Stop dist
  else ...

```

We also stop early if a full pass over the edge list does not update anything:

```

let is_dist_updated = List.fold_left (relax_edge dist) false edges in
if is_dist_updated then `Continue dist
else `Stop dist

```

So the core algorithm is:

```

let find_shortest_paths ~src edges =
  let dist = create_tbl ~src edges in
  ...
  List_utils.fold_until
    (relax_edges edges)
    Fun.id
    dist
    vertices

```

Predecessors Each table entry stores both the current shortest-known distance and the predecessor edge that produced it:

```
(distance, predecessor_edge)
```

The distance tells us the cost from `src`; the predecessor edge lets us reconstruct the path.

For the earlier graph, the shortest path to `a` is direct:

```

Minimum distance to "a" = 5
Predecessor edge of "a" is = r -> a (5)

```

The shortest path to 0^* goes through a :

Minimum distance to " 0^* " = 8

Predecessor edge of " 0^* " is $a \rightarrow 0^*$ (3)

Tracing predecessors gives:

$$r \rightarrow a \rightarrow 0^*$$

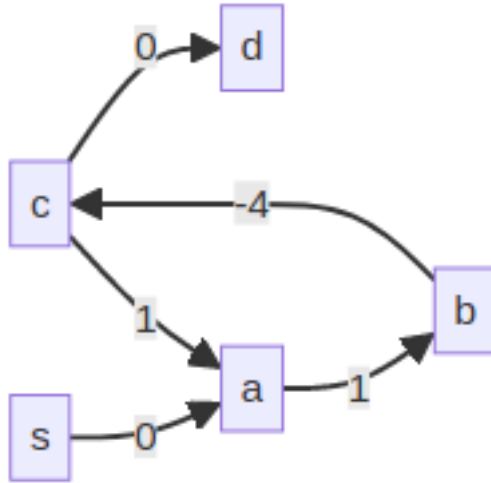
So one predecessor edge per node is enough to reconstruct a shortest path.

Detecting Negative Cycles After the normal relaxation loop, Bellman-Ford runs one extra pass over the edges. If any edge can still be relaxed, then the graph has a negative cycle.

```
let find_relaxed_node_opt edges dist =  
  List.find_map  
    (fun (_, to_, _) as edge) ->  
      if relax_edge dist false edge then Some to_  
      else None)  
  edges
```

The relaxed node proves a negative cycle exists, but it may not itself be inside the cycle. For example:

```
let relnode_not_in_neg_cycle =  
  [ ("c", "d", 0) ; ("s", "a", 0)  
    ; ("a", "b", 1) ; ("b", "c", -4)  
    ; ("c", "a", 1)  
  ] in  
print_mermaid_lr ~id:"relnode-not-in-neg-cycle-mermaid" relnode_not_in_neg_cycle;
```



Here, the first relaxed node can be d, even though d is only reached from the cycle:

Relaxed: d

To guarantee we land inside the cycle, we follow predecessor links `NUM_NODES` times. By the pigeonhole principle, this skips any non-cycle tail.

```

let find_cycle_entry_opt edges (tbl, num_nodes) =
  match find_relaxed_node_opt edges tbl with
  | None -> None
  | Some entry ->
    let rec move_back node n =
      if n = 0 then node
      else
        match find_predecessor node tbl with
        | None -> node
        | Some from_ -> move_back from_ (n - 1)
    in
    Some (move_back entry num_nodes)

```

Then we collect predecessor edges until we return to the start node:

```

let collect_cycle start (tbl, num_nodes) =
  let rec loop curr n acc =
    if n = 0 then acc
    else
      match find_predecessor_edge curr tbl with
      | None -> acc
      | Some ((from_, _, _) as pred_edge) ->
        let acc = pred_edge :: acc in

```

```

        if Node.compare from_ start = 0 then acc
        else loop from_ (n - 1) acc
    in
    loop start num_nodes []

```

The final bellman_ford function is:

```

let bellman_ford
  (type node)
  (module Node : Baby.OrderedType with type t = node)
  ~(src : node)
  (edges : node edge list)
  : [ `No_negative_cycle of (node * int) list
    | `Negative_cycle of node edge list
    ] =
  let open Make (Node) in
  let tbl = find_shortest_paths ~src edges in
  match find_cycle_entry_opt edges tbl with
  | None ->
    `No_negative_cycle (
      tbl
      |> Hashtbl.to_seq_keys
      |> Seq.map (fun node -> node, find_distance node tbl)
      |> List.of_seq
    )
  | Some entry ->
    `Negative_cycle (collect_cycle entry tbl)

```

In short:

1. Compute the distance table.
2. Run one extra relaxation pass.
3. If nothing changes, return the distances.
4. If something changes, backtrack predecessors and return the negative cycle.